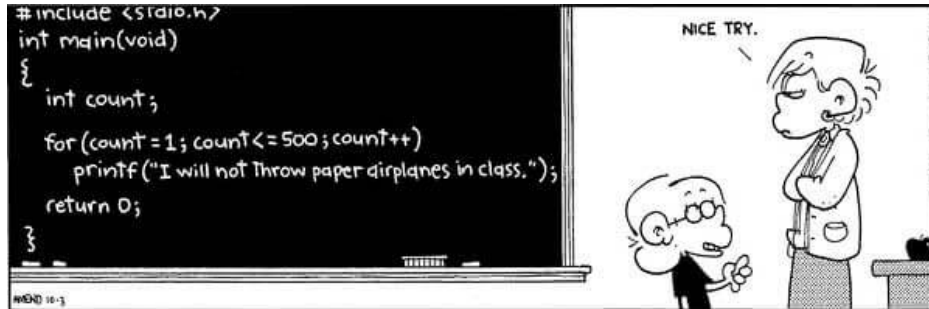


Autograded Homework 1: C Basics

CS 255: Computer Architecture and Machine Level Programming



- Each problem should be written in its own file with the name indicated in the specification.
- Each file should not contain a main function, though you will want one to test your functions while writing them.
- You may not use libraries except for `stdio.h` and `stdlib.h`. In particular, do not use `math.h`.
- You may assume the input provided will be correct, i.e. it will conform to the constraints.
- You may write any helper functions that you would like, as long as the functions that main will call have the signatures given in the problems exactly.
- To submit, you should upload five files named `minCoins.c`, `collatzSteps.c`, `dayOfWeek.c`, `isPalindrome.c`, and `computeStats.c` to Gradescope.

1. **minCoins** Consider the problem of trying to make change for a given amount of money using only U.S. coins. The machine has quarters (25¢), dimes (10¢), nickels (5¢), and pennies (1¢). You want to use the fewest possible coins.

For example, to make 37¢, one method is to use 37 pennies. But that takes 37 coins. A better method is to use one quarter (25¢), one dime (10¢), and two pennies (1¢ + 1¢), for a total of 4 coins. To make 99¢, the best method is three quarters (75¢), two dimes (20¢), and four pennies (4¢), for a total of 9 coins.

Your task is to write a function `minCoins` that returns the minimum number of coins needed to make exactly cents.

Specification:

filename: `minCoins.c`

```
int minCoins(int cents) {  
    \\ return the minimum number of coins needed to make exactly cents  
}
```

Constraints:

- `cents` is a non-negative integer

Examples:

- (a) `minCoins(0)` → 0
- (b) `minCoins(1)` → 1 (1 penny)
- (c) `minCoins(5)` → 1 (1 nickel)
- (d) `minCoins(37)` → 4 (1 quarter, 1 dime, 2 pennies)

2. **collatzSteps** The Collatz conjecture is a famous unsolved problem in mathematics. It starts with any positive integer n . If n is even, divide it by 2. If n is odd, multiply it by 3 and add 1. Repeat this process until you eventually reach 1. The conjecture states that no matter what number you start with, you will always reach 1.

Your task is to write a function `collatzSteps` that returns the length of the sequence from n to 1.

Specification

filename: `collatzSteps.c`

```
int collatzSteps(int n){  
    \\ return the length of the sequence from n to 1, inclusive  
}
```

Constraints

- n is a positive integer

Examples:

- (a) `collatzSteps(1)` $\rightarrow$ 1
- (b) `collatzSteps(2)` $\rightarrow$ 2 ($2 \rightarrow 1$)
- (c) `collatzSteps(5)` $\rightarrow$ 6 ($5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$)

3. **dayOfWeek** Days repeat on a 7-day cycle: Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday. Suppose we know that January 1, 2000 was a Saturday. Using this as a reference, we can compute the day of the week for any later date by counting days forward.

Your task is to write a function `dayOfWeek` that computes the day of the week for a given date in the Gregorian calendar.

Specification

filename: `dayOfWeek.c`

```
int dayOfWeek(int year, int month, int day){
    \\ return an integer (0-6) representing the day of the week
    \\ 0: Sunday
    \\ 1: Monday
    \\ 2: Tuesday
    \\ 3: Wednesday
    \\ 4: Thursday
    \\ 5: Friday
    \\ 6: Saturday
}
```

Constraints

- valid day, month, and year on or after January 1, 2000

Examples:

- (a) `dayOfWeek(2000,1,1)` → 6 (Saturday)
- (b) `dayOfWeek(2000,1,2)` → 0 (Sunday)
- (c) `dayOfWeek(2025,9,4)` → 4 (Thursday)
- (d) `dayOfWeek(2025,9,19)` → 5 (Friday)

4. **isPalindrome** A palindrome is a word, phrase, or sentence that reads the same forward and backward, ignoring differences in capitalization and punctuation.

For example:

- "racecar" is a palindrome.
- "A man, a plan, a canal: Panama" is also a palindrome, even though it includes spaces and punctuation.
- "hello" is not a palindrome.

Your task is to write a function `isPalindrome` that checks whether a string is a palindrome.

Specification

filename: `isPalindrome.c`

```
int isPalindrome(const char s[]){  
    \\ return 1 if s is a palindrome, 0 otherwise  
}
```

Constraints

- `s` is a null-terminated string containing only lowercase letters (a–z), digits (0–9), and spaces (' ')

Examples:

- (a) `isPalindrome("racecar")` → 1
- (b) `isPalindrome("no lemon no melon")` → 1
- (c) `isPalindrome("hello")` → 0

5. **computeStats** Given an array of integers, it is often useful to know simple statistics like the minimum, maximum, total sum, and average.

For example, the array [1, 2, 3, 4, 5] has:

- minimum = 1
- maximum = 5
- sum = 15
- mean = 3.0

Your task is to write a function `computeStats` that computes these values.

Specification

filename: `computeStats.c`

```
struct Stats {
    int min;
    int max;
    long sum;
    double mean;
};

struct Stats computeStats(const int a[], int n){
    \\ A struct Stats containing the minimum, maximum, sum, and mean.
}
```

Constraints

- `a` is an array of integers
- array length `n` is greater than 0

Examples:

- (a) `computeStats([1,2,3,4,5],5)` → min=1, max=5, sum=15, mean=3.0
(b) `computeStats([-2,-2,-2],3)` → min=-2, max=-2, sum=-6, mean=-2.0